SCHOOL OF COMPUTER SCIENCE & APPLIED MATHEMATICS

Computer Graphics and Visualisation

Group Project — a 3D browser game built with Three.js

COMS3006A · COMS3025A

! **Everything that changes from year to year lives on Moodle.** Group sign-up, submission links, exact dates, your allocated mentor and the venue for each demonstration are all published there. This brief describes what the project is, how it is marked, and how to get it running on the department server — none of which changes.

1 Overview

In this project you will work in a team of **4–6 students** to build a **3D computer game with three levels or stages** that runs in a web browser (Chrome on Ubuntu) using the **Three.js** graphics framework.

Your game should demonstrate mastery of the technical aspects of graphics covered in this course — for example hierarchical modelling, viewing and cameras, lighting and materials. Beyond those requirements you are free to design any 3D game you like, and you are encouraged to do something exciting and interesting.

```
// The three levels must be genuinely different
```

Three levels means three *distinct experiences*, not the same level three times with the furniture moved around. A new arrangement of the same obstacles, or the same arrangement with the numbers turned up, does not count as a second level.

Each level must **introduce or add something of its own**. That could be:

- a **new mechanic** — a new way of moving, a new tool or ability, a new way to fail;
- a **new environment or visual identity** — a different setting with lighting, materials and atmosphere to match;
- a **story or progression element** that moves the game forward and gives the player a reason to keep going;
- a **change in the kind of challenge** — for instance a level built around timing after one built around exploration, or a boss or set-piece encounter.

Be prepared to answer, for each level, the question **"what does this level do that the others do not?"** A game whose levels are interchangeable will lose marks under *Gameplay & Experience*, and cannot reach the top band for *Innovation*.

Marks are allocated to **polish** (have you put effort into the look and feel of your game beyond what HTML and Three.js give you by default) and to **innovation** (have you implemented something that is not obvious — networking, multi-player, special effects, novel models, animated models and so on).

Your final game must be **playable from the department's LAMP server**. See section 5 — read it early, because it constrains how you build the project, not just how you hand it in.

2 Three.js

Documentation, a live editor, a "Getting Started" guide and an extensive set of examples are at threejs.org. The textbook also discusses the library in Chapter 5.

Three.js provides a **scenegraph API**. You can load and build models, attach physics engines, lights and cameras to your scene, and the library traverses the graph to render it in much the same way as the examples shown in class. You can place lights and cameras in the world, or attach them as children of objects so that they move together.

Think carefully about the hierarchy of the objects in your world. Both the sensibility of your design and your motivation for it will be assessed — you should be able to explain why a given object is a child of another.

Three.js was chosen because it operates at a high enough level that you can create interesting and impressive games, but at a low enough level (relative to something like Unity) that you still have to demonstrate a grasp of

the design concepts taught in this course.

Note that Three.js is a JavaScript library and **all of your code executes in the browser on the player's machine**. You therefore have to be conscious of efficiency throughout — see section 6.

3 What you must deliver

Deliverable	Requirement
The game	A fully playable 3D game with three levels or stages, **each introducing something the others do not**, hosted on the department LAMP server and accessible directly through a web browser. It must run smoothly without any setup by the person playing it.
Trailer video	Maximum 2 minutes, uploaded to YouTube. Showcases the gameplay in an engaging, visually appealing way, similar to a commercial game trailer.
Devlog video	A behind-the-scenes explanation of the project: how you handled lighting and effects, how the gameplay mechanics were designed, and any original innovations your team implemented. Required for the final submission only.
Credits screen	Accessible from within the game, listing every library, asset and resource you did not create yourself — see below.
Contribution report	Submitted individually via Moodle — see section 9.

```
// Use whatever helps you build the game — then credit it
```

You may use **any external resource or library** that helps you build your game. There is no restricted list. That includes physics engines, Three.js add-ons and loaders, shader and post-processing libraries, UI and audio libraries, character and texture models, sound effects and music, and any tutorial, article, video or code sample you learn from or adapt. You are encouraged to search widely — Google, YouTube, OpenCourseWare, StackOverflow and the Three.js examples are all fair game, and relevant links are also posted on Moodle.

The one condition is that **everything you did not make yourself is acknowledged in the credits screen inside your game**: what it is, where it came from, and its licence where one applies. Credit anything you would feel awkward being asked about in the demonstration.

Properly credited third-party work is simply good engineering, and costs you nothing. Uncredited third-party work presented as your own is plagiarism, and is treated as such.

Bear in mind that *Innovation* rewards what **your team** contributed. Building on a physics library is expected; assembling a game entirely from other people's assets and code, however well credited, leaves little of your own for us to mark.

For examples of what strong submissions look like, see the **Hall of Fame** section of the CGV book.

4 Grading

You will be graded on the following aspects of your project. They will be assessed as is relevant to your game, but you should show a meaningful effort to illustrate each concept within your game.

Category	Weight	What is assessed
Viewing	**10%**	Are you able to load up a 3D scene? Is the scene animated? Can the user change the view in some way? Can the camera move through the world? Are there animation glitches? Is there a 3D avatar for the main character or other elements? Do you have objects that move with the world and items that move with the camera? Multiple views (first- and third-person, or an orthographic projection as a minimap)? Picture-in-picture?
Control & Playability	**10%**	Keyboard and mouse controls are required. Are they smooth, simple and logical for your game? Do they effectively control the scene, view and avatar as relevant? Does the game have an objective, and can the player succeed and fail at it? Is it competitive and fun? Is the game played in all three dimensions — you want to do more than a 2D platformer that looks 3D, so controls and movement should work in all three dimensions as relevant. Does the game have a working physics model?
3D Effects	**15%**	Demonstrate the graphical and 3D effects discussed in the course: antialiasing, depth tests, colour, multiple light sources, smooth and flat shading, curves, surfaces, static and dynamic skyboxes, shadows, reflections, refractions, and textures used for more than colour (bump maps, height maps). Using these in a visually appealing way contributes to polish; using them in interesting and unusual ways contributes to innovation. *Custom shader work is marked separately, below.*
Shaders	**10%**	Marked on its own, because writing your own stage of the programmable pipeline is a different skill from assembling the effects the framework already provides. Have you written your own vertex and fragment shaders rather than relying solely on built-in materials? Are uniforms, attributes and varyings used correctly, and are uniforms driven by time or game state so the effect is alive rather than static? Does the shader achieve something the built-in materials cannot — procedural texturing or noise,

Category	Weight	What is assessed
		vertex displacement, dissolves, toon or outline shading, water, force fields, post-processing passes? Is it integrated into the game rather than bolted on? **You must be able to explain your shader code and why it works** when you demonstrate.
Gameplay & Experience	**25%**	The overall quality of the player's experience — the creative and immersive aspects rather than the technical implementation alone. Does the game present a coherent storyline or theme? Are the graphics consistent and appealing, the controls smooth and intuitive? The depth, balance and fun of the gameplay are central, along with sound and music to enhance immersion. A strong project feels polished and engaging and has replay value; a weak one lacks cohesion, has frustrating controls, and fails to deliver a meaningful interaction.
Polish	**10%**	Impress us with the look and feel. Can the game restart without refreshing the page? Does it lag? Is there a dashboard or in-game menu? Is there a colour scheme? Quality-of-life features such as an options menu? Extra bells and whistles?
Innovation	**10%**	Impress us with new ideas. Try something different for your game concept, use your own models and textures designed in Blender, or use effects and techniques not covered in the course. Networking? Multi-player? Sound? Efficiency? Add the extra value that makes your game memorable.
Game Trailer	**10%**	How effectively the project is presented and promoted in a short video (max 2 min). It should highlight core features, showcase engaging gameplay moments, and reflect the theme or storyline in a polished, professional manner. Strong submissions use clear editing, pacing and sound or narration to create a cinematic presentation that could serve as a genuine marketing piece. At minimum, the trailer must demonstrate the game in action and give a coherent overview of the experience.

// Earning the top band

To earn the last 20% of the marks in the **Polish** and **Innovation** sections, your project needs something that makes it stand out as special. It should be memorable.

5 Hosting your game on the LAMP server

Your final game must be playable from the department's **LAMP** server. Markers will play it from a browser, at a URL, with no setup on their side — so getting this working is part of the project, not an afterthought.

// SSH access is no longer available

You cannot `ssh` or `scp` into the server, and you should not plan any part of your workflow around shell access to it. **Deployment happens by file transfer through Moodle.** You upload your built game as an archive to the submission set up for this purpose, and it is published to the server for you.

How to deploy

1. **Produce a deployment build** of your game — not your source tree. See section 6.3.
2. **Test that build locally** by serving the built folder over HTTP and playing it right through. This is the single most effective thing you can do to avoid a broken submission.
3. **Zip the contents of the build folder** so that `index.html` sits at the top level of the archive, not inside a nested directory.
4. **Upload the archive** to the Moodle submission. Follow the naming convention given there so your group's game lands in the right place.
5. **Open the published URL and play the whole game** in Chrome. Do this well before the deadline — the first upload very often reveals a path or case-sensitivity problem that never appeared on your own machine.

// Your game will be served from a subdirectory

Each group's game is published under its own folder, so your game will live at a URL of the form `https://<server>/<your-group-folder>/` and *not* at the root of the domain. Any path in your code that starts with `/` will therefore point at the wrong place. This is by far the most common reason a game that worked perfectly in development shows a blank screen once hosted — see section 6.2.

What the server will and will not do

- It serves **static files** over HTTP: HTML, JavaScript, CSS, models, textures, audio. That is all a Three.js game needs.
- It will **not** run your build tools. Node, npm, bundlers and dev servers exist only on your own machine. Whatever you upload is exactly what gets served.
- It will **not** install dependencies. Everything your game needs at runtime must be inside the archive you upload, or loaded from a public CDN over HTTPS.
- It runs **Linux**, which means filenames are case-sensitive.

6 Running in a browser: what to watch for

Your game is executed by the browser on the machine playing it, and it is served as ordinary files from a web server. Both facts have consequences that catch groups out every year. Budget time for them.

6.1 Resource constraints

A browser tab is not a games console. It has a limited memory budget, shares the GPU with everything else on the machine, and runs your code in a single main thread alongside rendering. Your game will be marked on lab hardware, not on your development laptop.

- **Budget for the marking machine, not yours.** If your game only holds a playable frame rate on a gaming laptop, it will lose marks under *Gameplay & Experience* and *Polish*, both of which explicitly penalise lag.
- **Textures dominate memory.** A handful of 4096×4096 textures will exhaust a modest GPU on their own. Scale textures down to the smallest size that still looks right, and prefer power-of-two dimensions. Compress source images — a 12 MB PNG that could have been a 400 KB JPEG costs you both memory and load time.
- **Watch your triangle count and your draw calls.** Many small meshes drawn separately are usually slower than one merged mesh. Reuse geometries and materials across objects instead of creating a new one per object.
- **Create nothing per frame.** Allocating vectors, materials or geometries inside your animation loop produces garbage-collection stutter. Create them once and reuse them.
- **Dispose of what you remove.** Removing a mesh from the scene does not free its GPU memory. Call `.dispose()` on geometries, materials and textures when you tear down a level, or memory will climb until the tab dies — which is exactly what happens during a three-level playthrough in front of a marker.
- **Shadows and post-processing are expensive.** Use shadow maps deliberately: limit which lights cast them, keep shadow map resolution sensible, and constrain the shadow camera to the area that actually needs it.
- **Keep total download size reasonable.** Everything must arrive over the network before play begins. Prefer `.glb` over `.gltf` with loose files, and consider Draco compression for large models. Add a loading screen — it is worth marks under *Polish* and it stops a slow first load from looking like a crash.
- **Profile rather than guess.** Chrome DevTools' Performance panel and a frame-rate counter will tell you where the time actually goes.

6.2 Relative versus absolute paths

Because your game is served from a subdirectory rather than the root of the server, every path in your project must be written **relative to the file that uses it**.

```
<!-- Absolute: resolves to https://server/assets/... and 404s -->
<script src="/src/main.js"></script>
loader.load("/assets/models/ship.glb", ...);

<!-- Relative: resolves inside your own folder, wherever it is published -->
<script src="./src/main.js"></script>
loader.load("./assets/models/ship.glb", ...);
```

A leading `/` means "start from the root of the whole server". It works during development because in development your game *is* at the root. Once published it points somewhere that does not exist, and you get a blank canvas with 404s in the console.

> **`// Linux filenames are case-sensitive`**
>
> Windows and macOS do not care whether you write `Rock_Texture.PNG` or `rock_texture.png`. The Ubuntu server does. A texture that loaded perfectly for three months on your laptop will silently fail to load once hosted, and your model will render untextured or black. Make the case in your code match the case on disk exactly, and keep asset filenames lowercase with no spaces to sidestep the problem entirely.

Two related traps:

- **Spaces and special characters in filenames** have to be URL-encoded and are a frequent source of confusing failures. Use `hyphen-separated-lowercase` names.
- **Mixed content.** If your page is served over HTTPS, any asset or library you pull from an `http://` URL will be blocked by the browser. Use HTTPS for every external resource.

6.3 Build a distribution version and upload that

What you develop with and what you deploy are not the same thing. A dev server (`npm run dev`) transforms your source on the fly — resolving bare module imports, bundling, and rewriting paths. None of that happens on the LAMP server, which simply hands out the files you gave it.

So **produce a production build and upload the output folder**. With Vite, the most common setup for a Three.js project:

```
# configure once, in vite.config.js, so asset URLs are relative:
export default { base: './' }

npm run build      # writes a self-contained site into dist/
npx serve dist     # serve the BUILD locally and play it through
                   # (not the dev server -- this is the point)
```

Then zip the *contents* of `dist/` and upload that.

> **// Do not upload your source tree**
>
> Uploading `node_modules/`, your `src/` folder and `package.json` does not produce a working game — it produces a directory listing and a lot of wasted upload. If your `index.html` contains `import * as THREE from 'three'` with no build step, the browser cannot resolve `'three'` and nothing will run.

If you are not using a bundler at all, that is perfectly acceptable — but then your project must already consist of plain files the browser can load directly:

- Use an **import map** in your `index.html`, or import Three.js from a full relative path such as `./libs/three.module.js`, or from an HTTPS CDN.
- Include the library files themselves in your upload if you are not using a CDN.
- Test by serving the folder over HTTP locally — for example `python3 -m http.server` — and never by opening `index.html` as a `file://` URL. Module scripts and texture loads are blocked under `file://`, so it tells you nothing useful about whether hosting will work.

> **// The one habit that prevents most deployment failures**
>
> Deploy early and deploy often. Get an empty scene with one textured cube hosted and playing from the server in the first week or two, then keep it updated. A group that first uploads the night before the deadline is a group that discovers all of the above at the worst possible moment.

7 Groups and mentors

You should work in groups of **4 to 6**, captured on Moodle. Each group is allocated a **mentor** from the pool of CGV tutors.

During each lab session you should meet with your mentor for a few minutes to ask questions and get advice and ideas. **It is your responsibility to approach your mentor** with the questions you have.

8 Assessment stages

You have three opportunities for feedback.

8.1 Alpha — formative, not for marks

You should have Three.js up and running, with a preliminary implementation you can use to show your mentor what the game will ultimately look like. Your mentor will walk through the rubric with you, and you need to be able to answer questions about how you have implemented — or plan to implement — the required functionality for each aspect of it. The beta and final assessments follow the same format.

8.2 Beta — graded

The beta version is graded against the rubric by a combination of tutors, and counts as the **beta project mark** towards your class mark.

You will demonstrate your project in action, talk us through the game, and answer questions about your design choices, special effects and so on. It is acceptable for the project to have minor bugs at this stage, or for a level or two to be incomplete, but it should fundamentally be finished and almost ready for production. **This is your last chance for formal feedback before the final hand-in.** The trailer is included at this stage; the devlog is required only for the final submission.

8.3 Final release — graded

Deliver a fully playable version of your game **hosted on the LAMP server**, running smoothly and accessible directly through a web browser, together with the trailer and devlog videos described in section 3.

9 Individual and group assessment

The project is assessed as a group. However, as with group work in Software Design, each group member submits information via Moodle about what their own and others' responsibilities were, what they contributed to the project, and in what proportions they think the marks should be awarded within the group.

Your individual mark may be adjusted by up to 20% based on the assessment of these results. If you do not contribute, your grade will be adjusted.

10 Schedule

Milestone	When
Groups finalised on Moodle	**1 week** from the release of this brief
Alpha — concept and preliminary implementation shown to your mentor	The lab session in the week after groups are finalised. Be ready to explain your game idea and how you intend to meet each of the grading criteria. Formative, not for marks
Beta demonstration	**2 weeks** after groups are finalised — that is, 3 weeks from the release of this brief. Graded, and counts towards your class mark
Final submission — hosted game, trailer, devlog	End of the teaching term; the last day on which changes to your submission are accepted

```
// This is a short runway — start immediately
```

There are only **three weeks between this brief and the graded beta**. Do not spend the first week waiting for a group to form around you: sign up, agree a concept, and get a Three.js scene running in the first few days. Get something hosted on the server early (section 5) rather than leaving deployment until the end, because deployment problems are the ones that cost whole days.

! **Exact dates are published on Moodle** and take precedence over anything written here. The milestones themselves do not change from year to year.

11 Rubric

The following rubric acts as a guide for how the project will be marked.

Category	A (80–100%)	B (70–79%)	C (60–69%)	D (50–59%)	E (40–49%)	F (<40%)
Viewing (10%)	Full 3D scene with smooth camera controls, multiple view modes (first/third person, minimap), no glitches, complex animated avatars and environment elements.	3D scene loaded and animated with smooth single camera mode and minor view options, very few glitches.	Basic 3D scene, camera moves in world, some animation but limited interactivity or polish.	Scene loads but camera control is awkward or restricted, minimal animation, occasional glitches.	Scene barely loads, camera mostly fixed, animations broken or absent.	Scene fails to load correctly or camera does not work at all.
Control & Playability (10%)	Intuitive, responsive keyboard and mouse controls supporting all intended mechanics fluidly. Clear objectives and win/loss states, fun and challenging gameplay in all three dimensions with relevant physics.	Mostly intuitive and responsive controls with minor stiffness. Clear objectives and win/loss states, 3D gameplay mostly functional, physics mostly appropriate.	Functional controls but occasionally unresponsive or unintuitive mapping. Basic objectives, limited gameplay depth, physics implemented but not central.	Controls partially work but are inconsistent or laggy. Objectives unclear or incomplete, physics inaccurate or irrelevant.	Controls barely functional or missing key gameplay actions. Gameplay barely functional, minimal objectives, 3D aspect largely unused.	No functional player control; game unplayable or no objective implemented.
3D Effects (15%)	Multiple advanced effects (lighting, shadows, reflections, skybox, etc.) implemented creatively and in visually appealing ways. Application of textures for enhanced 3D effects (bump map, height map, etc.)	Several advanced effects used appropriately with good visual results.	Some effects present but limited in variety or quality.	Minimal effects implemented, basic shading only. Basic application of textures for model colouring.	Single simple effect with little visual impact.	No noticeable graphical effects beyond default rendering.
Shaders (10%)	Several custom shaders written by the team, with both vertex and fragment stages used purposefully. Uniforms driven by time or game state. Achieves effects the built-in materials cannot, integrated into the game's look and mechanics. Team can explain the code fully.	At least one substantial custom shader with correct use of uniforms, attributes and varyings, producing a good visual result and clearly understood by the team.	A working custom shader of limited scope — simple colour manipulation or basic vertex displacement — with modest visual impact.	Minimal shader work: a small modification of an example shader, or a single hard-coded fragment colour, with limited understanding shown.	A shader is present but broken, visually negligible, or reproduced without comprehension.	No custom shader work; built-in materials only.
Gameplay & Experience (25%)	Highly engaging experience: polished storyline or theme, consistent and appealing graphics, smooth controls with no lag both visually and when inputting commands, balanced and fun gameplay,	Good experience: clear theme or story, mostly consistent graphics, smooth controls with few issues or minor lag,	Adequate experience: basic theme or loose storyline, graphical consistency acceptable but unrefined, controls functional,	Weak experience: unclear or poorly executed theme, inconsistent visuals, clunky controls, gameplay repetitive or	Very poor experience: little cohesion in graphics, controls frustrating, gameplay dull or broken, no meaningful	Unplayable: no coherent storyline or theme, broken visuals, controls non-functional, gameplay absent, no sound design.

Category	A (80–100%)	B (70–79%)	C (60–69%)	D (50–59%)	E (40–49%)	F (<40%)
	immersive sound/music, and strong replay value. Each level introduces a genuinely new mechanic, setting or story beat that builds on the last.	gameplay enjoyable with minor flaws, decent sound design. Levels are clearly distinct from one another.	gameplay has some fun moments but lacks depth, sound minimal. Levels differ, but mostly in layout or difficulty.	confusing, weak or missing sound design. Game is very laggy but still playable.	sound or feedback.	
Polish (10%)	Highly polished: no lag, restart/menu systems, consistent colour scheme, extra features that enhance experience. Quality-of-life features such as option menus and optimisation.	Well-polished with minimal lag, restart/menu, mostly consistent design.	Some polish present (e.g. menu or restart), occasional lag, basic visual consistency.	Rough presentation, lag present, few extra features.	Very rough, frequent lag, poor visual coherence.	No polish, broken presentation, unplayable state.
Innovation (10%)	Original ideas, unique mechanics, custom assets, possible multiplayer or novel effects/techniques.	Some originality and custom work beyond core requirements.	Minor originality, small custom elements.	Mostly standard template game with little deviation.	Minimal originality, almost entirely default assets.	No originality, direct copy of tutorial/demo.
Game Trailer (10%)	Trailer is highly professional, cinematic, and engaging; showcases gameplay clearly with excellent editing, pacing, sound, and narration; effectively markets the game.	Trailer is polished and engaging, clearly showcases gameplay, good editing and sound use, minor areas for improvement.	Trailer demonstrates gameplay adequately, editing and pacing are basic but functional, communicates the game.	Trailer is rough, gameplay footage present but poorly edited, sound or pacing issues.	Trailer is very poor, minimal editing, unclear gameplay presentation.	No trailer submitted or completely unusable.

12 Final submission checklist

- ☐ Game has **three levels or stages** and can be played from start to finish.
- ☐ Each level **introduces something the others do not** — and you can say in one sentence what that is.
- ☐ Keyboard *and* mouse controls both work.
- ☐ At least one **custom shader** is in the game, and every member can explain what it does.
- ☐ The game can be **restarted without refreshing the page**.
- ☐ Credits screen lists **everything you did not make yourself** — libraries, models, textures, sounds, music, and any code or tutorial you adapted — with sources and licences.
- ☐ **Production build** created — not the source tree, not `node_modules`.
- ☐ Built game **tested locally over HTTP**, played all the way through.

- [ ] No absolute paths beginning with `/` anywhere in the code or markup.
- [ ] Asset filenames match the case used in the code, exactly.
- [ ] Archive uploaded via Moodle, with `index.html` at its top level.
- [ ] **Published URL opened in Chrome and played through**, with the browser console checked for 404s.
- [ ] Frame rate is acceptable on lab hardware, and memory does not climb across the three levels.
- [ ] Trailer (max 2 min) uploaded to YouTube and the link submitted.
- [ ] Devlog video submitted.
- [ ] Every member has submitted their individual contribution report on Moodle.

School of Computer Science & Applied Mathematics · University of the Witwatersrand, Johannesburg. Dates, submission links and group allocations are published on Moodle and take precedence over this document.